

LINUX CHMOD COMMANDS

chmod = change file permissions

Permissions:

- r = read
- w = write
- x = execute

Users:

- u = owner/user
 - g = group
 - o = others
 - a = all
-

Basic Syntax

chmod [permissions] file_name

Example:

chmod +x script.sh

Makes file executable.

Symbolic Mode Commands

Add permission

Add execute permission

chmod +x file.sh

Add read permission for all

chmod a+r file.txt

Add write permission for owner

chmod u+w file.txt

Add execute permission for group

```
chmod g+x script.sh
```

Remove permission

Remove execute permission

```
chmod -x file.sh
```

Remove write permission from others

```
chmod o-w file.txt
```

Remove all permissions for others

```
chmod o-rwx file.txt
```

Set exact permissions

Owner full, group read, others none

```
chmod u=rwx,g=r,o= file.txt
```

Numeric Mode (Most Common)

Number	Permission	Meaning
4	r	Read
2	w	Write
1	x	Execute

Numbers:

- 7 = rwx
- 6 = rw-
- 5 = r-x

- 4 = r--
- 0 = ---

Format:

owner group others

Common chmod numbers

Full access to owner only

`chmod 700 file.sh`

Owner full, others read/execute

`chmod 755 file.sh`

Very common for scripts/directories.

Owner read/write, others read only

`chmod 644 file.txt`

Very common for normal files.

Everyone full access

`chmod 777 file.txt`

No permissions

`chmod 000 file.txt`

Check Current Permissions

`ls -l`

Example output:

```
-rwxr-xr-- 1 user user 1234 file.sh
```

Meaning:

- owner = rwx
 - group = r-x
 - others = r--
-

Permission Cheat Sheet

Number	Permission
7	rwx
6	rw-
5	r-x
4	r--
3	-wx
2	-w-
1	--x
0	---

LINUX GREP COMMAND

What is grep?

grep is used to search text or patterns inside files.

Syntax: grep [options] "pattern" filename

Search word in file : grep "error" log.txt

Important

Option	Detailed Description	Example
-h	Hides the filename while displaying matching lines. Useful when searching in multiple files and you only want the matched content.	grep -h "error" *.txt
-c	Counts how many lines contain the matching pattern instead of displaying the actual lines.	grep -c "error" file.txt
-l	Displays only the names of files that contain the matching pattern. It does not show the matching lines.	grep -l "error" *.txt
-v	Displays all lines that do NOT match the given pattern. Used to exclude specific text.	grep -v "error" file.txt
-o	Displays only the matched part of the line instead of printing the entire line.	grep -o "[0-9]*" file.txt
-e	Allows searching for multiple patterns at the same time. Multiple -e options can be used together.	grep -e "cat" -e "dog" file.txt
-i	Ignores uppercase and lowercase differences while searching. Makes the search case-insensitive.	grep -i "hello" file.txt
-n	Displays line numbers along with matching lines. Helpful for locating text inside large files.	grep -n "error" file.txt

Optional

Option	Detailed Description	Example
-r	Searches recursively through all files and subdirectories.	grep -r "main" .
-w	Matches only the complete word, not partial words.	grep -w "cat" file.txt
-x	Matches the entire line exactly with the pattern.	grep -x "hello" file.txt
-A	Displays specified number of lines after the matching line.	grep -A 2 "error" log.txt
-B	Displays specified number of lines before the matching line.	grep -B 2 "error" log.txt
-C	Displays specified number of lines before and after the match.	grep -C 2 "error" log.txt
-E	Enables extended regular expressions for advanced pattern matching.	`grep -E "cat
--color	Highlights matched text with color in output for easier visibility.	grep --color "error" file.txt
-s	Suppresses error messages such as “file not found”.	grep -s "hello" missing.txt
-m	Stops searching after a specified number of matches.	grep -m 3 "error" file.txt